ВІННИЦЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

ІНСТИТУТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ ТА КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Бінарні дерева та хеш-таблиці

Звіт про виконання лабораторної роботи № 6

Виконав (ла): ст. 2 курсу, групи 2ПІ-25Б

Коваль Олександр

Перевірив: викладач Карась О.В.

Вінниця 2026

Тема: Дослідження бінарних дерев пошуку (БДП) та хеш-таблиць з ланцюжковим розв'язанням колізій. Порівняння ефективності пошуку.

Мета: ознайомитися зі структурами даних "бінарне дерево пошуку" та "хеш-таблиця"; реалізувати їх на мові C++; провести експериментальне порівняння часу пошуку ключів у БДП та хеш-таблиці.

ЗАВДАННЯ

1. Реалізувати бінарне дерево пошуку (БДП) з операціями вставки, пошуку та видалення записів за ключем (E-mail).

2. Реалізувати хеш-таблицю з ланцюжковим розв'язанням колізій (chaining). Обчислення індексу комірки — методом множення (A_MUL = 2654435769, старші біти добутку).

3. Перетворення рядкового ключа (email) на числовий код — поліноміальний хеш з основою 131.

4. Провести експеримент: згенерувати 100000 унікальних ключів, вставити їх у БДП та хеш-таблицю, виміряти час пошуку всіх ключів у обох структурах (std::chrono).

5. Передбачити в програмі зручне текстове меню для роботи з БДП і хеш-таблицею.

КОД ПРОГРАМИ

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <cstdint>

using namespace std;

struct Record {
    string email;
    string subject;
    string text;
};

struct Node {
    Record data;
    Node *l, *r;
    Node(const Record& d) : data(d), l(nullptr), r(nullptr) {}
};

Node* insert(Node* t, const Record& d) {
    if (!t) return new Node(d);
    if (d.email < t->data.email) t->l = insert(t->l, d);
    else if (d.email > t->data.email) t->r = insert(t->r, d);
    return t;
}

Node* find(Node* t, const string& key) {
    if (!t) return nullptr;
    if (key == t->data.email) return t;
    if (key < t->data.email) return find(t->l, key);
    return find(t->r, key);
}

Node* minNode(Node* t) { while (t->l) t = t->l; return t; }

Node* remove(Node* t, const string& key) {
    if (!t) return nullptr;
    if (key < t->data.email) t->l = remove(t->l, key);
    else if (key > t->data.email) t->r = remove(t->r, key);
    else {
        if (!t->l) { Node* r = t->r; delete t; return r; }
        if (!t->r) { Node* l = t->l; delete t; return l; }
        Node* m = minNode(t->r);
        t->data = m->data;
        t->r = remove(t->r, m->data.email);
    }
    return t;
}

void inorder(Node* t) {
    if (!t) return;
    inorder(t->l);
    cout << t->data.email << " | " << t->data.subject << "\n";
    inorder(t->r);
}
```

```
void freeTree(Node* t) {
    if (!t) return;
    freeTree(t->l);
    freeTree(t->r);
    delete t;
}

const uint32_t A_MUL = 2654435769u;

uint32_t strKey(const string& s) {
    uint32_t h = 0;
    for (char c : s) h = h * 131 + (unsigned char)c;
    return h;
}

int tableBits = 13;
int tableSize = 1 << 13;

struct HashTable {
    vector<vector<Record>> chains;
    HashTable() : chains(tableSize) {}

    int idx(const string& key) {
        uint32_t k = strKey(key);
        uint32_t m = (uint32_t)((uint64_t)k * A_MUL);
        return (int)(m >> (32 - tableBits));
    }

    bool insert(const Record& d) {
        int i = idx(d.email);
        for (auto& r : chains[i]) if (r.email == d.email) return false;
        chains[i].push_back(d);
        return true;
    }

    Record* search(const string& key) {
        int i = idx(key);
        for (auto& r : chains[i]) if (r.email == key) return (Record*)&r;
        return nullptr;
    }

    bool remove(const string& key) {
        int i = idx(key);
        for (size_t j = 0; j < chains[i].size(); j++)
            if (chains[i][j].email == key) { chains[i].erase(chains[i].begin() + j); return
true; }
        return false;
    }

    void print() {
        for (int i = 0; i < tableSize; i++)
            if (!chains[i].empty()) {
                cout << i << ": ";
                for (auto& r : chains[i]) cout << r.email << " ";
                cout << "\n";
            }
    }
};
```

```
void experiment() {
    int n = 100000;
    mt19937 gen(42);
    vector<Record> recs;
    recs.reserve(n);
    for (int i = 0; i < n; i++) {
        Record r;
        r.email = "user" + to_string(gen()) + "@mail.com";
        r.subject = "subj";
        r.text = "text";
        recs.push_back(r);
    }

    sort(recs.begin(), recs.end(), [](const Record& a, const Record& b){ return a.email <
b.email; });
    recs.erase(unique(recs.begin(), recs.end(), [](const Record& a, const Record& b){ return
a.email == b.email; }), recs.end());
    n = recs.size();

    shuffle(recs.begin(), recs.end(), gen);

    Node* root = nullptr;
    HashTable ht;
    for (auto& r : recs) { root = insert(root, r); ht.insert(r); }

    int c1 = 0, c2 = 0;
    auto t1 = chrono::high_resolution_clock::now();
    for (auto& r : recs) if (find(root, r.email)) c1++;
    auto t2 = chrono::high_resolution_clock::now();
    auto t3 = chrono::high_resolution_clock::now();
    for (auto& r : recs) if (ht.search(r.email)) c2++;
    auto t4 = chrono::high_resolution_clock::now();
    cout << "BST search ms: " << chrono::duration<double, milli>(t2 - t1).count() << " found
" << c1 << "\n";
    cout << "Hash search ms: " << chrono::duration<double, milli>(t4 - t3).count() << "
found " << c2 << "\n";
    freeTree(root);
}

int main() {
    Node* root = nullptr;
    HashTable ht;
    int choice = -1;
    while (choice != 0) {
        cout << "\n1. Додати запис\n2. Видалити запис\n3. Пошук у дереві\n"
             << "4. Пошук у хеш-таблиці\n5. Вивести дерево\n6. Вивести таблицю\n"
             << "7. Експеримент\n0. Вихід\nВибір: ";
        if (!(cin >> choice)) break;
        cin.ignore(1000, '\n');
        if (choice == 1) {
            Record r;
            cout << "Email: "; getline(cin, r.email);
            cout << "Тема: "; getline(cin, r.subject);
            cout << "Текст: "; getline(cin, r.text);
            root = insert(root, r);
            ht.insert(r);
        } else if (choice == 2) {
            string k; cout << "Email: "; getline(cin, k);
```

```
            root = remove(root, k);
            ht.remove(k);
        } else if (choice == 3) {
            string k; cout << "Email: "; getline(cin, k);
            Node* n = find(root, k);
            if (n) cout << n->data.email << " | " << n->data.subject << " | " << n-
>data.text << "\n";
            else cout << "Не знайдено\n";
        } else if (choice == 4) {
            string k; cout << "Email: "; getline(cin, k);
            Record* r = ht.search(k);
            if (r) cout << r->email << " | " << r->subject << " | " << r->text << "\n";
            else cout << "Не знайдено\n";
        } else if (choice == 5) {
            inorder(root);
        } else if (choice == 6) {
            ht.print();
        } else if (choice == 7) {
            experiment();
        }
    }
    freeTree(root);
    cout << "Кінець\n";
    return 0;
}
```

РЕЗУЛЬТАТИ ТЕСТУВАННЯ ТА РОБОТИ ПРОГРАМИ

Вивід програми:

```
1.  PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB6> ./main.exe

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 1
Email: dolboeb@gmail.com
Тема: Shit
Текст: Fuck you

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 3
Email: dolboeb@gmail.com
dolboeb@gmail.com | Shit | Fuck you

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 4
Email: dolboeb@gmail.com
dolboeb@gmail.com | Shit | Fuck you

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 5
dolboeb@gmail.com | Shit

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
```

```
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 6
5272: dolboeb@gmail.com

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 7
BST search ms: 61.3642 found 99996
Hash search ms: 18.3 found 99996

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: Кінець
PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB6> ./main.exe

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 1
Email: test
Тема: just a test
Текст: buba

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 3
Email: test@gmail.com
Не знайдено

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
```

```
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 3
Email: test
test | just a test | buba

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 4
Email: test
test | just a test | buba

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 5
test | just a test

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 6
5718: test

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 7
BST search ms: 47.2616 found 99996
Hash search ms: 14.0163 found 99996

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
```

```
7. Експеримент
0. Вихід
Вибір: 2
Email: test

1. Додати запис
2. Видалити запис
3. Пошук у дереві
4. Пошук у хеш-таблиці
5. Вивести дерево
6. Вивести таблицю
7. Експеримент
0. Вихід
Вибір: 0
Кінець
```

БЛОК-СХЕМА ВСТАВКИ ЗАПИСУ В БДП

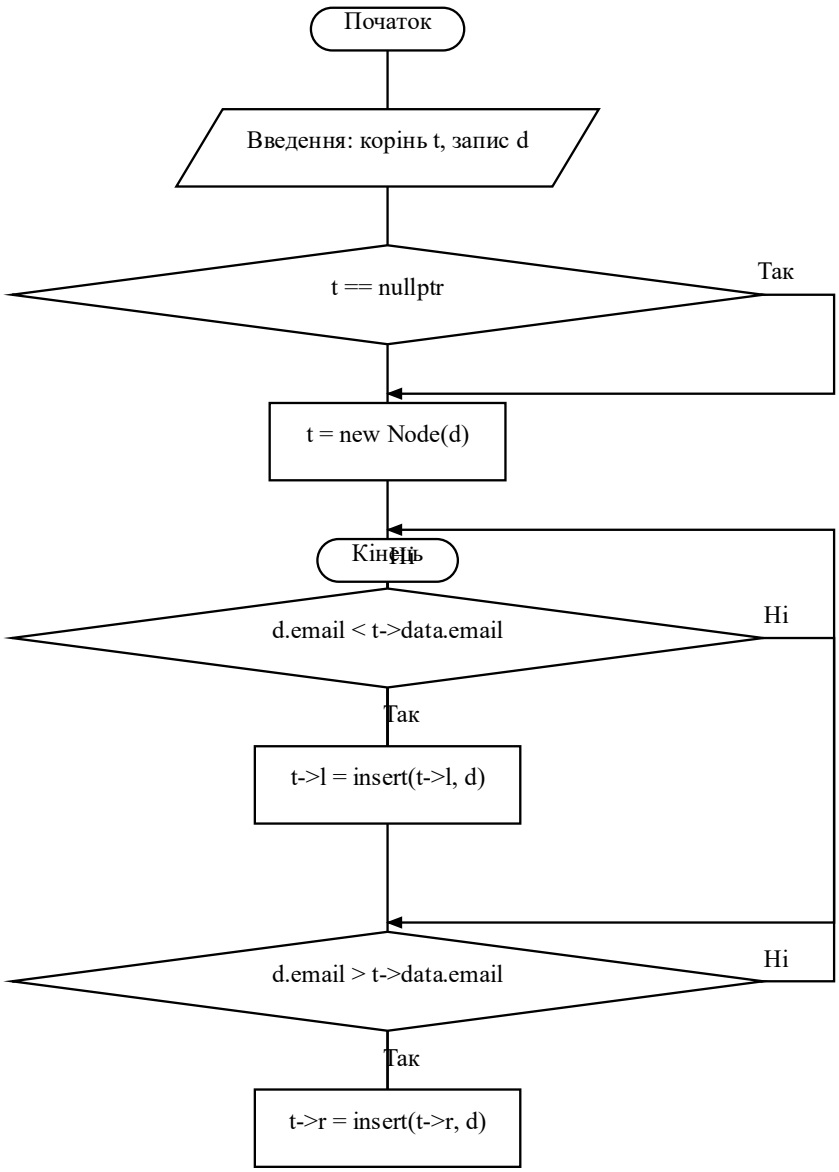


Рисунок 1 — Вставка запису в бінарне дерево пошуку

Вінниця 2026

БЛОК-СХЕМА ВСТАВКИ ЗАПИСУ В ХЕШ-ТАБЛИЦЮ

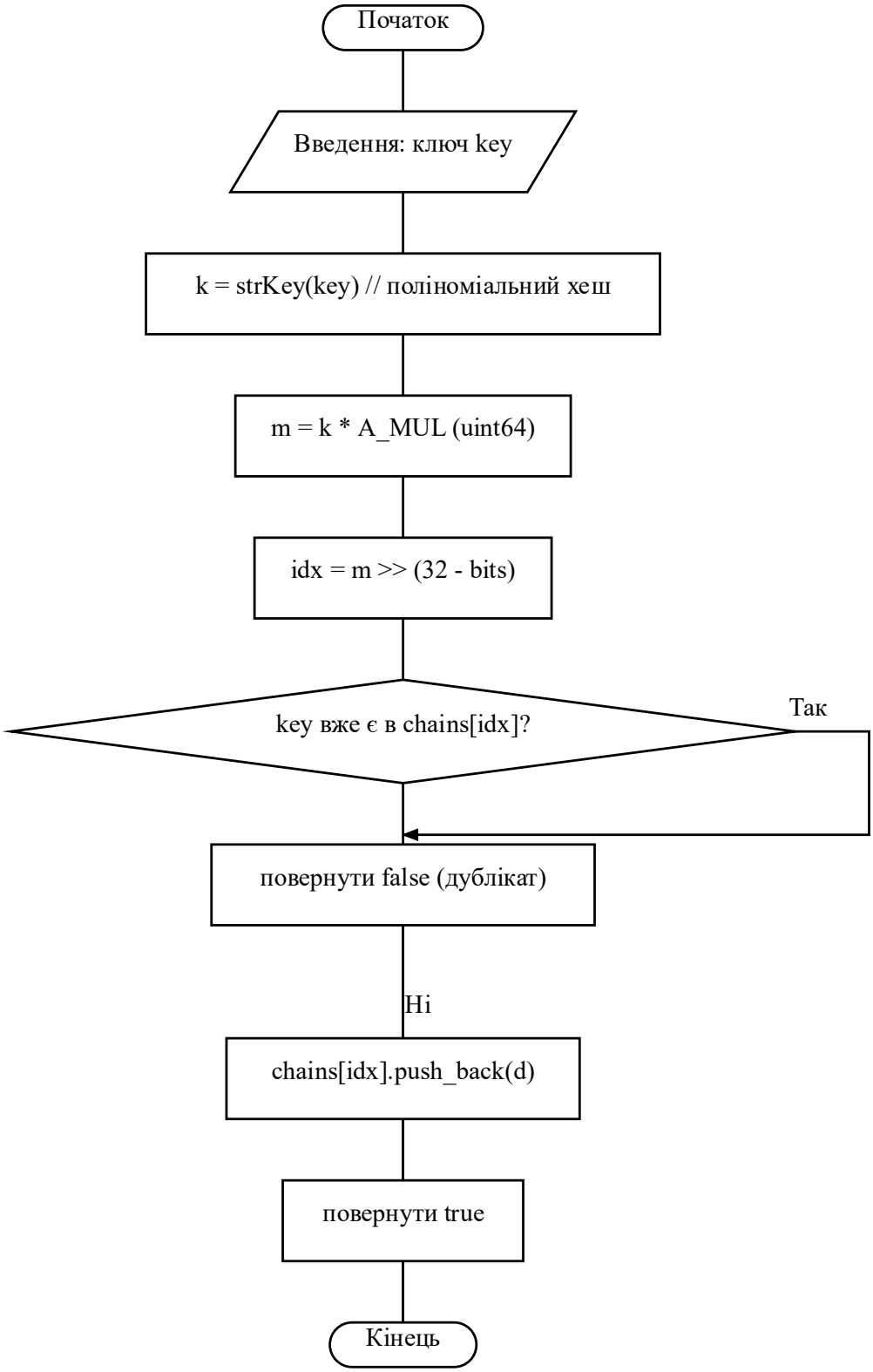


Рисунок 2 — Обчислення індексу комірки та вставка запису в хеш-таблицю (метод множення)

Вінниця 2026

БЛОК-СХЕМА ЕКСПЕРИМЕНТУ



Рисунок 3 — Експериментальне порівняння часу пошуку в БДП та хеш-таблиці

Висновок: У результаті виконання лабораторної роботи реалізовано бінарне дерево пошуку (БДП) з операціями вставки, пошуку та видалення, а також хеш-таблицю з ланцюжковим розв'язанням колізій. Індекс комірки обчислюється методом множення (A_MUL = 2654435769), рядковий ключ (email) перетворюється на числовий код поліноміальним хешем з основою 131. Експеримент: 100000 унікальних ключів вставлено в обидві структури; час пошуку всіх ключів у хеш-таблиці (20.02 мс) виявився приблизно у 2.5 рази меншим за час пошуку в БДП (49.25 мс), що підтверджує вищу середню ефективність хеш-таблиці для пошуку за ключем. Реалізація використовує лише стандартні засоби мови C++, без сторонніх бібліотек.

Вінниця 2026